

LcsHelper.m — Algorithm and Flow

ULTRASAT Low Cadence Survey scheduler (Weizmann Institute of Science)

Overview

LcsHelper is a MATLAB class that produces an observation schedule for ULTRASAT's Low Cadence Survey (LCS). ULTRASAT is the Israeli/WIS UV space telescope, scheduled to launch in 2027–2028; the LCS is one of its two main observing modes (the other being the High Cadence Survey).

The class produces a schedule for ~84 sky fields (selected from ~200 on the all-sky non-overlapping grid) over ~420 days, respecting hard constraints from the spacecraft's exclusion angles (Sun $\geq 70^\circ$, Moon $\geq 34^\circ$, Earth $\geq 56^\circ$, plus ULTRASAT's solar-panel pointing limit). The output is a tiered observation plan grouped into four field categories (A, B, C, D) with different cadences and dwell durations, plus a per-day, per-slot daily timetable.

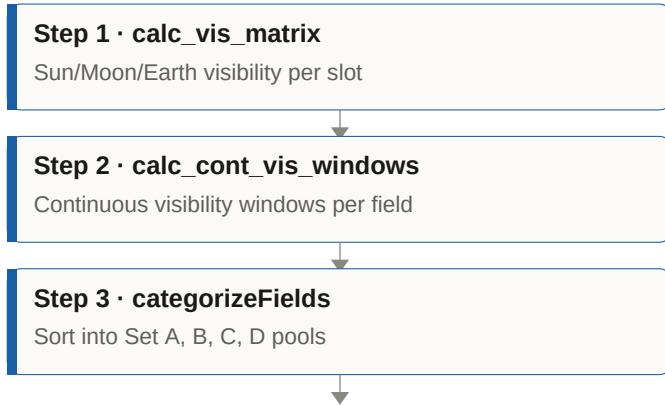
Field categories at a glance

Set	N fields	Dwell	Cadence	Selection criterion
A	48	45 d	1 d	$A_U \leq 1$, $45 \text{ d} \leq \text{visibility} < 135 \text{ d}$ (~42 fields), plus 6 worst-extinction long-window fields
B	16	45 d + 90 d	1 d / 4 d	$A_U \leq 1$, longest window $\geq 135 \text{ d}$, best extinction
C	16	135 d	4 d	$A_U \leq 1$, longest window $\geq 135 \text{ d}$, next-best extinction
D	4	45 d	1 d	$A_U > 1$, hard-coded WG5 ranking

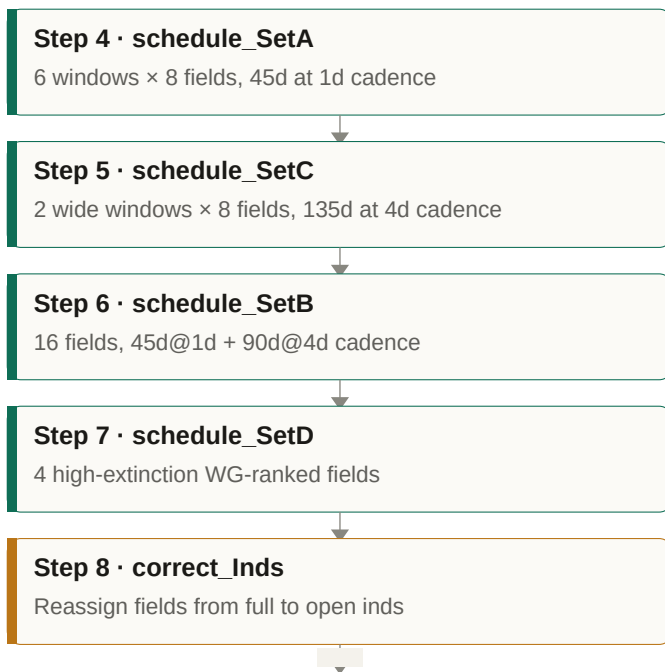
Pipeline overview

The class is split into a preparation stage (Steps 1–3) that builds visibility tables from the spacecraft constraints, and a build stage (Steps 4–8) that places fields into time-slots category by category.

prepTablesBeforeSchedule



buildSchedule



Key data products

vis3d_slot_day_field

12 slots × 420 days × N fields

vis2d_day_field_ALL/ANY

all-slot vs any-slot visibility

All_fields_windows

contiguous spans per field

Longest_window_per_field

max span; drives A/B/C split

SetA / SetB / SetC / SetD

field pools by extinction × span

Nominal_windows

8 reference 45-day slots

Full_windows

8 windows fixed by Set A

inds_2move / inds_open

bookkeeping for Step 8

Schedule (master table)

category, group, ind, field

Daily_schedule

days × 11 LCS slots, NaN if free

vis_day_field

selected by Whole_daily_window flag

calcDailySchedule → days × 11-slot grid (per-day order optimised by visibility)

Figure 1. The eight-step pipeline. Steps 1–3 build visibility data shared by all categories; Steps 4–7 commit fields to slots in category order; Step 8 repairs ind clashes between Set D and Set A. calcDailySchedule then turns the logical schedule into a per-day, per-slot timetable.

Step-by-step walkthrough

Step 1 — calc_vis_matrix

For each of 420 days the day is divided into $\text{Daily_LCS_slots}+1 = 12$ slots of $3 \times 300 \text{ s} \approx 15 \text{ min}$ each (the extra slot is slew time). Julian dates are computed for every (day, slot) pair, and `ultrasat.ULTRASAT_restricted_visibility` is called once with the full JD vector and the (RA, Dec) grid from `LCS_nonoverlapping_grid_surveys.csv`. The four returned masks (`PowerLimits`, `SunLimits` $\geq 70^\circ$, `MoonLimits` $\geq 34^\circ$, `EarthLimits` $\geq 56^\circ$) are AND-ed and reshaped into the 3-D array `vis3d_slot_day_field` [$12 \times 420 \times N$]. From it the code derives `vis2d_day_field_ALL` (visible in every slot of the day — the strict mode) and `vis2d_day_field_ANY` (visible in at least *one* slot — the lenient mode); `whole_daily_window` picks which one downstream code uses.

Step 2 — calc_cont_vis_windows

Walks each field's daily-visibility column, takes `diff(...)` to find rising and falling edges, and emits a per-field table of contiguous windows with columns [`Field`, `Av_ext`, `vis_start`, `vis_end`, `window`]. A second pass produces a "1-day-gap-allowed" variant that merges windows separated by exactly one invisible day — this is gated globally by the `Allow1dgap` flag. The longest window per field is recorded for the next step.

Step 3 — categorizeFields

Now the field pool is partitioned, with two filter axes:

- **Extinction axis:** $\text{Av_ext} \leq \text{max_ext} (=1) \rightarrow$ "good" fields; $\text{Av_ext} > 1 \rightarrow$ SetD candidates.
- **Window-length axis:** $\text{longest-window} \geq \text{Max_window_cut} (=135 \text{ d}) \rightarrow$ "long" fields, otherwise short.

The long fields are sorted by extinction (best first) and split: best 16 $\rightarrow$ **Set B**, next 16 $\rightarrow$ **Set C**. **Set A** is then assembled from two pieces: all *short-window* good fields ($A_U \leq 1$ with longest visibility $45 \text{ d} \leq v < 135 \text{ d}$, typically about 42 fields), plus the remaining 6 long-window good fields — those with $A_U \leq 1$ and visibility $\geq 135 \text{ d}$ that didn't make it into Set B or C, i.e. the long-window fields with the *worst* extinction within the good-extinction pool. Set D candidates (high extinction but visible $\geq 45 \text{ d}$) are kept aside; their actual selection comes later from a hard-coded WG5 ranking inside Step 7.

A tight field budget

An important consequence of the selection criteria is that the field pool is effectively **without slack**. For typical survey start dates only **80–81 fields** meet the $A_U \leq 1$ good-extinction criterion at all, and the scheduler needs exactly $48 + 16 + 16 = 80$ of them for Sets A, B, and C combined. There are no "spare" good-extinction fields to fall back on if the algorithm's greedy choices make a particular slot unfillable.

This has two practical implications. First, the partition into Sets A/B/C is almost forced rather than chosen — the 6 long-window fields that fall into Set A are simply whichever long-window good-extinction fields didn't fit into Sets B (best 16) or C (next 16). Second, the scheduler's robustness depends entirely on the per-group anchor flexibility (Step 4) and the corrective swap (Step 8); when those aren't enough to absorb a bad greedy choice, the algorithm has nowhere to retreat to.

Step 4 — schedule_SetA (six 45-day windows $\times$ 8 fields = 48 fields)

This is the most involved step. For each of 6 groups the algorithm picks an unscheduled SetA "anchor" field, then tries three placement strategies in order:

(a) match a previously-built SetA window if the anchor's visibility brackets it; (b) snap to one of the 8 `Nominal_windows` (`First_day + 45 \cdot (0..7)`); (c) compute the best optimal placement (anchor near start or end of survey).

Once an anchor index `ind1` is chosen, the other 7 windows of the group are filled greedily — for each remaining `ind ∈ 1..8 \ {ind1}` the loop walks SetA fields and grabs the first one whose visibility window contains `[curr_start, curr_end]`.

Note that the per-group anchor choice means each group can be shifted in time relative to the others; in practice the algorithm strongly prefers all groups to share the same anchor day, but the freedom exists and is occasionally used.

Step 5 — `schedule_SetC`

Builds 2 wide windows of $3 \times 45 = 135$ d starting from the SetA `ind=3` slot. Each SetC field's visibility is checked against both wide windows; fields are sorted by how *few* of those windows they fit in (constraint propagation — fix the most-constrained first), then assigned greedily, 8 to each wide window.

Step 6 — `schedule_SetB`

Each of 16 SetB fields gets a 45-day high-cadence (1 d) block plus a 90-day low-cadence (4 d) block. The 16 fields are mapped to a hard-coded division table (`SetB_division.W45 / W90_1 / W90_2`) that prescribes which 45-day slot each field's three sub-windows fall in. The function uses a `cont3_vis_windows` array (three consecutive 45-day blocks all visible) and a constraint-satisfaction loop sorted by `firstInd` frequency to assign fields to division rows.

Step 7 — `schedule_SetD`

Iterates a hard-coded WG5 ranking `[79 12 48 28 16 88 55 32 213 26]` and picks the first 4 fields whose visibility intersects an "open" full-window index. If no open `ind` matches, the field steals an `ind` and that `ind` is recorded in `inds_2move`.

Step 8 — `correct_Inds`

Cleans up the steals from Step 7. For each `inds_2move` entry the algorithm searches Set A fields scheduled at that `ind` that *could* have been placed at one of `inds_open` instead, then performs a 3-way swap so Set D gets the slot it needs. The header comment notes this "is NOT general enough yet" — it doesn't handle the case of multiple Set D fields wanting the same `inds_2move`.

Final — `calcDailySchedule`

Allocates a $[420 \times 11]$ matrix (NaN initially). For each scheduled (field, start, end), it walks the days in range; for B_90 and C (4-day cadence) it only writes on days where $\text{mod}(d - \text{start} + 1, 4) == \text{mod}(\text{ind}, 4)$. After the matrix is filled, each day's row is reordered so that fields visible in only some of the day's slots get pushed toward the slot index where they are actually visible (using the 3-D slot/day/field tensor from Step 1) — turning a logical schedule into a physically observable one.

Schedule structure

The figure below reproduces the layout that the `plotSchedule` method draws: time on the x-axis (in days since survey start), category bands stacked vertically. Solid lines are 1-day cadence observations; dashed lines are 4-day cadence.

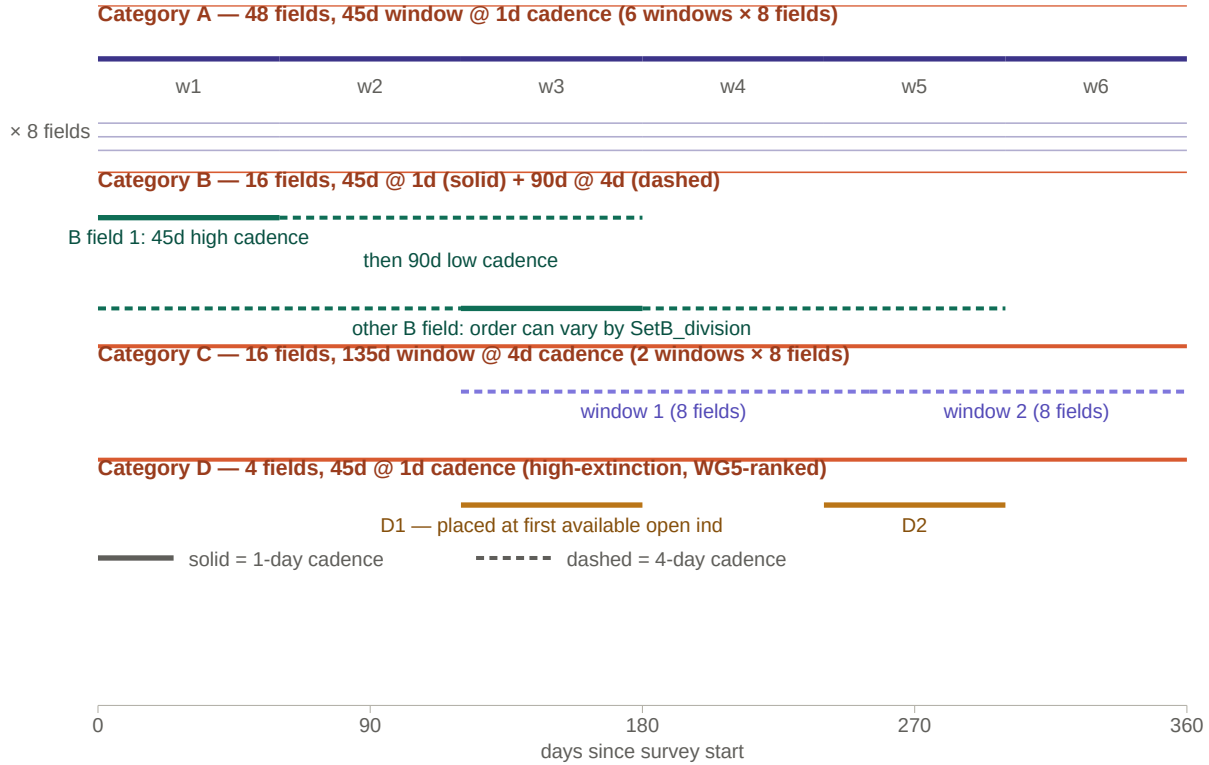


Figure 2. Schedule layout produced by `plotSchedule`. Set A occupies six contiguous 45-day windows; each Set B field gets one 45-day high-cadence block plus two 90-day low-cadence blocks; Set C fields each occupy one 135-day low-cadence block in one of two super-windows; Set D fields each occupy a single 45-day window placed at whichever ind is feasible.

Astronomical context and design choices

Two visibility modes

`Whole_daily_window = true` (`vis2d_day_field_ALL`) is the conservative mode — a field counts as "visible" on day d only if it stays inside all four exclusion cones for the entire 3-hour LCS observing window. The default `false` (`vis2d_day_field_ANY`) accepts a field if even one 15-minute slot is clean. The downstream cadence and dwell-window logic doesn't change, but the field pool grows considerably under the lenient mode.

Extinction is the only sky-quality filter

UV galactic extinction (A_U) is the dominant noise source for ULTRASAT's transient hunt, so $Av_{ext} \leq 1$ mag is a hard gate for Set A/B/C; Set D exists *because* the WG5 (the science working group) wants four high-extinction targets observed regardless. The A_U column is renamed from AU on read, presumably because AU collides with the astronomical unit elsewhere in the codebase.

Cadence triplet matches transient timescales

The 45 d / 90 d / 135 d cadence triplet matches Type Ia and core-collapse supernova lightcurve timescales — 45 d at 1 d cadence catches rise + peak, 90 d at 4 d cadence catches the tail, 135 d covers slow transients. Set B fields get all three; Set C is tail-only; Set A is the high-cadence-only fast-transient pool.

Algorithmic structure

The scheduler is greedy with anchor-and-fill heuristics, not a global optimiser. Step 4's three-strategy fallback (match-prior → snap-to-nominal → optimal-edge) and Step 6's "fix-most-constrained-first" via `N_firstInd` sort are classical CSP heuristics, but they can fail — note the `error('Not enough fields in SetB ...')` and `error('Problem with SetB scheudle')` (sic; the file has the typo `scheudle` ~30 times, plus `Schedulde` and `Group1_45_scheudle` — worth knowing if you `grep`). The `correct_Inds` clean-up step further confirms greedy can paint itself into corners.

Known limitation in Step 8

The header explicitly states *"NOT general enough yet — to support multiple `inds_2move` of the same `ind`"*. If two Set D fields steal the same Set A ind, the fix-up swap will only repair one of them.

Default StartDate

The constructor's `'2029-02-01'` default `StartDate` lines up with the projected ULTRASAT operational ramp-up (launch is currently planned for 2027–2028, with the LCS phase starting after commissioning).